INSTITUTO TECNOLOGICO SUPERIOR PROGRESO

CARRERA:
INGENIERIA EN SISTEMAS COMPUTACIONALES

GRADO Y GRUPO:
9no - Grupo 1

MATERIA:
PENTESTING

TAREA:
Reporte de Investigación: Pentesting

FECHA DE ENTREGA:
15 de septiembre de 2026

ESTUDIANTE:
Carlos Eduardo Valencia Hernández

DOCENTE:
MARTÍNEZ GARCÍA HOLZEN



Contenido

Reporte de Análisis de Seguridad 2
1. Resumen Ejecutivo 4
2. Descripción del Escenario Analizado 4
3. Metodología 4
4. Identificación de Vulnerabilidades 4
5. Análisis de Riesgos 5
6. Propuesta de Soluciones (Mitigación) 6
7. Tabla de Vulnerabilidades vs Mitigación (OBLIGATORIO) 6
8. Conclusiones 7





1. Introducción

Hoy en día, la seguridad informática es uno de los temas más críticos para cualquier empresa o institución. Con la llegada de los servicios en la nube, el trabajo remoto y las aplicaciones web modernas, las redes corporativas ya no están protegidas por un simple perímetro físico. Al mismo tiempo, los ataques cibernéticos se han vuelto mucho más frecuentes y sofisticados: desde grupos organizados que buscan dinero mediante ransomware hasta actores avanzados que intentan robar información confidencial o sabotear operaciones.

Durante mucho tiempo, la respuesta tradicional de las organizaciones fue adoptar una postura netamente defensiva y reactiva: instalar antivirus, colocar cortafuegos (firewalls) y esperar que las alertas avisaran cuando algo saliera mal. Sin embargo, los constantes incidentes de seguridad que vemos en las noticias demuestran que esta estrategia es insuficiente. El defensor tiene que proteger miles de puertos, servidores y usuarios, mientras que al atacante le basta con encontrar un solo error o una mala configuración para comprometer toda la red.

Frente a esta realidad surge la necesidad de la seguridad ofensiva y, de manera concreta, del pentesting o pruebas de penetración. La idea central es muy simple pero contundente: la mejor forma de saber si un sistema es seguro es intentar vulnerarlo de forma autorizada y controlada, pensando y actuando exactamente como lo haría un atacante real antes de que un criminal lo haga.

El objetivo de este informe es analizar a fondo el pentesting desde tres ángulos clave: su fundamentación teórica y metodológica, su evolución histórica y su relación con casos reales de la industria, y finalmente una reflexión crítica sobre los éxitos, fracasos y lecciones que nos dejan estos ejercicios en el mundo real. Con esto se busca entender no solo las herramientas técnicas, sino también el valor estratégico, los límites operativos y las buenas prácticas que deben seguirse para que estas pruebas realmente mejoren la seguridad de los sistemas.





2. Aspecto teórico: fundamentos del pentesting

2.1 Definición clara de pentesting

El test de penetración (pentesting) es un método de evaluación de seguridad en el cual un profesional o equipo especializado simula un ataque real contra una infraestructura tecnológica, red o aplicación, bajo autorización previa y por escrito. Su meta fundamental no es únicamente encontrar fallos teóricos en el software, sino intentar explotarlos de manera controlada para verificar hasta dónde podría llegar un atacante y qué impacto real tendría esa intrusión en la organización.

En la práctica y en los cursos de seguridad, suele existir confusión entre tres actividades que parecen similares pero tienen enfoques y alcances muy distintos:

- **Escaneo de vulnerabilidades:** Es un proceso casi 100% automatizado mediante software (como Nessus u OpenVAS) que rastrea puertos y servicios para detectar fallos conocidos en bases de datos (firmas). Aunque es rápido, genera muchos falsos positivos y no prueba si una falla realmente se puede explotar en el entorno real.
- **Auditoría de cumplimiento:** Es una revisión de tipo administrativo y documental para comprobar si una empresa cumple con normas o leyes específicas (como ISO 27001, PCI-DSS para tarjetas de crédito o GDPR). Funciona como una lista de verificación (checklist) donde se certifica que existen ciertas políticas, pero no se prueba la resistencia técnica real del sistema.
- **Pentesting (prueba de penetración):** Es un ejercicio práctico, manual y analítico. El especialista utiliza herramientas pero sobre todo su propio razonamiento para encadenar fallos menores (exploit chaining), evadir defensas, escalar privilegios en los servidores y demostrar con pruebas fehacientes qué datos o sistemas quedaron expuestos, descartando por completo los falsos positivos.

2.2 Objetivos y alcances

Los objetivos principales de un pentesting son: identificar las vías de acceso que un atacante podría aprovechar, cuantificar el impacto operacional o económico que causaría una intrusión, evaluar la





efectividad real de las defensas y de los equipos de monitoreo (Blue Team), y entregar a la dirección técnica recomendaciones claras para solucionar los problemas detectados.

Dependiendo del nivel de información previa que la empresa evaluada entregue a los analistas, el pentesting se clasifica en tres modalidades reconocidas:

- **Caja negra (Black Box):** Los analistas no reciben ninguna información técnica interna, solo el nombre de la empresa o dominios públicos. Simula el punto de vista de un atacante externo desconocido. Requiere una fase muy intensa de recolección de información (OSINT) y escaneo inicial.
- **Caja blanca (White Box):** El equipo de pruebas recibe información completa: diagramas de arquitectura de red, código fuente, documentación de APIs e incluso credenciales con distintos niveles de privilegio. Es ideal cuando se desea una revisión profunda y exhaustiva del sistema.
- **Caja gris (Grey Box):** Es el esquema más habitual en las empresas. Se entrega un acceso limitado, como una cuenta de usuario estándar y nociones básicas de la red. Simula el escenario sumamente común de un empleado interno descontento o de un atacante que ya consiguió engañar a un usuario mediante phishing y busca moverse por la red interna.

Un componente indispensable en cualquier prueba son las **Reglas de Compromiso (Rules of Engagement o RoE)**. Este es el contrato formal y legal que delimita exactamente qué se puede y qué no se puede hacer durante la prueba. En las RoE se definen las direcciones IP autorizadas, los servidores excluidos (para no afectar la operación diaria o servidores de cobro), los horarios permitidos para las pruebas, y los protocolos de aviso inmediato si se descubre una vulnerabilidad extremadamente grave. Sin este documento firmado por la autoridad competente, cualquier intento de intrusión constituye un delito informático.

2.3 Metodologías reconocidas

Para que un pentesting sea profesional, ordenado y repetible, no se trabaja de forma improvisada, sino siguiendo metodologías estándar reconocidas por la industria de ciberseguridad:

- **PTES (Penetration Testing Execution Standard):** Es una de las metodologías más completas del ámbito profesional. Define siete etapas sucesivas: 1) Acuerdos iniciales (Pre-engagement), 2) Recolección de información (Intelligence Gathering), 3) Modelado de





amenazas (Threat Modeling), 4) Análisis de vulnerabilidades, 5) Explotación, 6) Post-explotación (donde se mide el valor de los datos comprometidos y persistencia), y 7) Elaboración del reporte técnico y ejecutivo.

- **NIST SP 800-115:** Desarrollada por el Instituto Nacional de Estándares y Tecnología de EE. UU. Es la guía de referencia técnica para entidades de gobierno y organizaciones que manejan infraestructuras críticas. Organiza las pruebas en cuatro grandes fases: Planificación, Descubrimiento, Ataque y Reporte.
- **OSSTMM (Open Source Security Testing Methodology Manual):** Diseñada por ISECOM, aporta un enfoque científico y métrico. Se enfoca en cuantificar la seguridad operacional calculando Valores de Evaluación de Riesgo (RAV) y analizando cómo interactúan las personas, la infraestructura tecnológica y la seguridad física.
- **OWASP WSTG (Web Security Testing Guide v4.2):** Es el estándar indiscutible para auditar aplicaciones y servicios web. Contiene guías paso a paso organizadas en 12 categorías para probar la autenticación, autorización, gestión de sesiones, validación de datos de entrada, criptografía y errores en la lógica de negocio de las páginas y APIs.

3. Aspecto histórico: evolución del pentesting

3.1 Orígenes del pentesting

El origen del pentesting se remonta a la década de 1960, cuando la expansión de los sistemas de tiempo compartido (time-sharing) hizo evidente que varios usuarios podían acceder simultáneamente a un mismo computador, lo que abrió interrogantes sobre la confidencialidad de la información. En 1967, durante la Joint Computer Conference, especialistas en seguridad debatieron por primera vez de forma pública la necesidad de "penetrar" deliberadamente los sistemas para descubrir sus debilidades antes de que lo hiciera un tercero no autorizado.

A partir de esa discusión, el Departamento de Defensa de los Estados Unidos, la Fuerza Aérea, la NSA y la RAND Corporation impulsaron la creación de los llamados "tiger teams": grupos formados por especialistas que intentaban comprometer sistemas gubernamentales y militares bajo autorización explícita. Paralelamente, en 1970, Willis H. Ware coordinó para la RAND Corporation el informe Security Controls for Computer Systems, donde se alertó formalmente





sobre la fragilidad de los enlaces remotos. En 1971 la Fuerza Aérea de EE. UU. ordenó pruebas de seguridad sobre sistemas de tiempo compartido, y en 1974 se realizó una de las evaluaciones más citadas de esta etapa: la prueba de penetración al sistema operativo MULTICS, que reveló fallas críticas de seguridad y sirvió de base para investigaciones posteriores sobre arquitecturas de referencia seguras.

Un hito fundamental ocurrió en 1972, cuando James P. Anderson publicó un reporte técnico que proporcionó lineamientos paso a paso para los Tiger Teams, estableciendo las bases metodológicas del pentesting moderno. Los pasos formalizados por Anderson fueron:

1. Encontrar una vulnerabilidad explotable.

2. Diseñar un ataque alrededor de ella.

3. Probar el ataque en un entorno controlado.

4. Aprovechar una línea en uso para operaciones ACS (acceso al canal del sistema).

5. Introducir el ataque en el objetivo.

6. Explotar la entrada para la recuperación o modificación de información clasificada.

Poco después, en 1973, el Departamento de Defensa formalizó la primera metodología de prueba denominada FMH (Flaw Hypothesis Methodology), estructurando el proceso de plantear hipótesis sobre fallos lógicos, verificarlos de forma activa y documentar el impacto para su corrección.

3.2 Evolución de técnicas y herramientas

Durante los años 80 y 90, con la masificación de las redes corporativas e Internet, el pentesting pasó de ser una práctica exclusivamente militar a una necesidad del sector privado. Es en esta etapa cuando el término "hacking ético" comienza a popularizarse para distinguir la actividad autorizada de la intrusión maliciosa. En sus comienzos, las pruebas requerían técnicas manuales de phreaking y war dialing mediante software como ToneLoc para discar miles de teléfonos buscando módems corporativos desprotegidos, además de scripts en C y Perl desarrollados de forma artesanal.

Hacia finales de los 90, las técnicas manuales dieron paso a herramientas especializadas: SATAN (1995) fue el primer escáner que generó debate público sobre la detección remota; Nmap (creado en 1997 por Fyodor) revolucionó el escaneo de puertos y el reconocimiento de sistemas operativos; y Nessus (1998) automatizó el rastreo de vulnerabilidades en servicios de red.





A mediados de los 2000 surgieron frameworks integrales como Metasploit (2003), que unificó la explotación de vulnerabilidades conocidas en una plataforma modular y extensible, cambiando para siempre el trabajo de los analistas. En el ámbito de las aplicaciones web, la aparición de Burp Suite y OWASP ZAP facilitó la interceptación y manipulación de tráfico HTTP, mientras que la publicación periódica del OWASP Top 10 (desde 2003) estandarizó el lenguaje sobre los riesgos más comunes en el software.

En la década de 2010, la sofisticación de las amenazas avanzadas persistentes (APT) impulsó la aparición de plataformas de simulación de adversarios como Cobalt Strike, Mythic o Sliver, orientadas no solo a encontrar vulnerabilidades técnicas aisladas sino a emular cadenas completas de ataque (kill chain), incluyendo movimientos laterales y persistencia silenciosa. Más recientemente, la disciplina ha incorporado automatización basada en inteligencia artificial para priorizar hallazgos y correlacionar información de reconocimiento a gran escala (OSINT), a la par del auge del pentesting como servicio (PTaaS) y de los programas de recompensas por vulnerabilidades (Bug Bounty), que amplían la superficie de escrutinio con comunidades globales de investigadores.

3.3 Relación con casos reales

La evolución del pentesting no puede desligarse de los incidentes reales que han demostrado, en la práctica, tanto el valor de las pruebas de penetración como las graves consecuencias de omitirlas o aplicarlas de forma superficial:

- **Brecha de Target (2013):** Atacantes ingresaron a la red corporativa a través de credenciales robadas a un proveedor de climatización (Fazio Mechanical) que tenía acceso remoto. Debido a una deficiente segmentación de red interna, los atacantes se movieron lateralmente hasta alcanzar los terminales punto de venta (POS) y robar datos de más de 40 millones de tarjetas de pago. Este caso evidenció la importancia de evaluar la seguridad de terceros dentro del alcance de cualquier pentest.
- **Brecha de Equifax (2017):** La explotación de una vulnerabilidad conocida en Apache Struts (CVE-2017-5638), para la cual ya existía parche oficial disponible meses antes, expuso datos personales y financieros de 147 millones de personas. El caso ilustra que detectar una falla





técnica o recibir un reporte no basta si la organización no tiene un proceso ágil para gestionar su remediación.

- **Brecha de Capital One (2019):** Una atacante explotó una mala configuración en un firewall de aplicación web (WAF) alojado en Amazon Web Services (AWS) mediante una vulnerabilidad SSRF (Server-Side Request Forgery). A través de esto accedió a los metadatos de la nube y robó datos de más de 100 millones de clientes, mostrando los riesgos específicos de las arquitecturas cloud y la necesidad de auditar permisos IAM.
- **Brecha de Uber (2022):** Un atacante obtuvo acceso a los sistemas internos de la empresa mediante técnicas de ingeniería social y fatiga de autenticación multifactor (MFA fatigue) dirigida a un empleado externo contratista. Una vez dentro de la intranet, encontró credenciales administrativas en scripts compartidos. Esto demostró que las pruebas de penetración deben incorporar de forma sistemática vectores humanos y no limitarse a lo puramente tecnológico.
- **Colonial Pipeline (2021):** El compromiso de una contraseña filtrada de una cuenta de VPN que no tenía autenticación multifactor provocó la paralización del mayor gasoducto de combustibles de la costa este de EE. UU., subrayando la enorme relevancia del pentesting en sectores de infraestructura crítica.
- **Caso SolarWinds (2020):** La inserción de código malicioso en las actualizaciones oficiales de la plataforma Orion demostró el peligro de los ataques a la cadena de suministro de software (SUNBURST), dejando claro que los pentests modernos deben auditar también los pipelines de desarrollo e integración continua (CI/CD).

4. Aspecto reflexivo: análisis crítico

4.1 Casos exitosos analizados

Entre los ejemplos que demuestran el valor del pentesting bien ejecutado se encuentran los programas de Bug Bounty de compañías como Google, Microsoft o el propio Departamento de Defensa de EE. UU. (con iniciativas como 'Hack the Pentagon'). En estos esquemas, investigadores externos identifican y reportan de forma responsable fallos críticos antes de que puedan ser explotados maliciosamente, permitiendo a los equipos de desarrollo corregirlos oportunamente.





De igual manera, los ejercicios de Red Teaming en el sector bancario alineados a marcos como TIBER-EU han demostrado una alta efectividad para exponer brechas en la capacidad de detección y respuesta de los equipos internos. Al simular atacantes reales, estos ejercicios suelen descubrir configuraciones erróneas críticas en Active Directory (como delegaciones Kerberos indebidas o elevación de privilegios locales) que los escáneres automáticos pasan por alto por completo.

El éxito de estos casos tiene en común un único elemento esencial: la madurez en el proceso organizacional. Esto implica contar con un alcance bien definido, un seguimiento formal de los hallazgos, la asignación de responsables técnicos para la remediación y la repetición periódica de las pruebas para verificar que las brechas fueron efectivamente solucionadas.

4.2 Casos fallidos analizados

En contraste, casos como el de Equifax muestran que la existencia de hallazgos de seguridad no garantiza protección si la organización carece de un proceso eficaz de gestión de parches e inventario de activos. La empresa realizaba auditorías periódicas, pero el reporte técnico quedó archivado sin aplicarse en los servidores productivos correctos.

De forma análoga, incidentes como el de Uber en 2022 o su fuga de datos de 2016 (donde credenciales de AWS estaban expuestas en repositorios privados de GitHub) evidenciaron que muchas organizaciones concentran sus pruebas exclusivamente en la infraestructura perimetral oficial y descuidan el factor humano y los entornos de desarrollo, a pesar de que la ingeniería social continúa siendo el vector de compromiso inicial más efectivo.

Asimismo, en el plano operativo se han documentado casos donde pruebas de penetración mal planificadas o sin reglas de compromiso claras causaron la caída involuntaria de servidores de producción o el bloqueo de autómatas programables (PLCs) en redes industriales y sistemas SCADA, demostrando que un pentest sin la debida calibración puede causar el mismo daño que intenta prevenir.

4.3 Reflexión sobre lecciones aprendidas

Al investigar los casos exitosos y fallidos se desprenden lecciones muy concretas sobre las limitaciones del modelo tradicional de seguridad:





- **El pentesting es una fotografía puntual:** Representa el estado de seguridad en una fecha y hora específicas. En empresas modernas que actualizan código y despliegan servicios varias veces por semana, el valor de un informe anual se diluye con rapidez si no se complementa con evaluaciones continuas.
- **La trampa del cumplimiento de papel:** Realizar pruebas únicamente para obtener un sello de certificación reglamentario genera una falsa sensación de seguridad. Las organizaciones suelen limitar artificialmente el alcance de la prueba para 'aprobar la auditoría', dejando fuera los sistemas más expuestos.
- **La remediación es tan importante como la detección:** Descubrir cien fallas no mejora la seguridad de la empresa si el equipo técnico no cuenta con el tiempo, los recursos o el conocimiento necesario para solucionarlas. Un pentesting exitoso termina con los parches aplicados, no con la entrega del documento PDF.
- **El factor humano no se puede omitir:** Ningún firewall o sistema de cifrado detiene a un atacante que engaña a un empleado mediante vishing o fatiga de notificaciones push. La seguridad técnica debe complementarse siempre con programas de concientización y pruebas controladas de ingeniería social.

4.4 Propuestas de buenas prácticas

Para superar estas deficiencias y lograr que las pruebas ofensivas aporten valor real, se proponen las siguientes buenas prácticas para las organizaciones actuales:

- **Adoptar ejercicios de Purple Teaming:** Fomentar que el equipo ofensivo (Red Team) y el defensivo (Blue Team) trabajen juntos en tiempo real durante la prueba. Esto permite a los defensores ajustar sus reglas en el SIEM o antivirus inmediatamente después de que un ataque es probado.
- **Integración continua en DevSecOps (Shift-Left):** Incorporar pruebas automatizadas de código (SAST y DAST) en las fases iniciales del ciclo de desarrollo. Esto soluciona los errores básicos de forma temprana y permite que los analistas humanos se concentren en buscar fallas complejas de lógica en el pentest final.





- **Enfoque de pruebas guiadas por inteligencia de amenazas (TLPT):** Diseñar los ejercicios ofensivos basándose en amenazas reales documentadas que atacan específicamente al sector de la empresa, como lo exige el marco TIBER-EU para instituciones bancarias.
- **Gestión de vulnerabilidades por riesgo real (RBVM):** Priorizar la solución de fallas considerando la probabilidad de explotación real (métricas como EPSS y el catálogo de exploits conocidos de CISA) en lugar de guiarse únicamente por el puntaje numérico del CVSS.
- **Definición exhaustiva de Reglas de Compromiso (RoE):** Establecer límites claros, contactos de emergencia y salvaguardas operativas para proteger la estabilidad de los servicios en producción.





5. Conclusiones

A través de este informe se ha examinado cómo el pentesting ha dejado de ser una actividad puramente experimental de laboratorios militares para convertirse en un componente indispensable de la ingeniería de sistemas moderna. La historia de los primeros Tiger Teams en los años 70 y la evolución de herramientas como Nmap, Metasploit y las modernas plataformas de emulación de adversarios muestran que la disciplina ha evolucionado al mismo ritmo acelerado que la tecnología y los métodos de los atacantes.

Sin embargo, la principal conclusión que se extrae de los incidentes reales analizados es que la efectividad del pentesting no depende de la sofisticación de las herramientas empleadas, sino de la madurez operativa de la organización. Casos como Target, Equifax o Uber confirman que una prueba técnica aislada no previene incidentes si no existe un compromiso real para corregir los hallazgos, si se excluyen partes críticas de la infraestructura o si se ignora el comportamiento humano.

En definitiva, el pentesting no debe verse como un trámite para aprobar una auditoría, sino como un proceso de aprendizaje continuo. Cuando se combina con enfoques colaborativos como Purple Teaming y se integra en el desarrollo ágil mediante DevSecOps, el hacking ético se convierte en la mejor herramienta con la que cuentan los profesionales de sistemas para anticiparse a los incidentes y construir sistemas verdaderamente confiables y resilientes.





6. Referencias

Anderson, J. P. (1972). *Computer security technology planning study* (Technical Report ESD-TR-73-51, Vol. I & II). Electronic Systems Division, Air Force Systems Command, United States Air Force. https://apps.dtic.mil/sti/citations/AD0758206

Herzog, P. (2010). *OSSTMM 3: The Open Source Security Testing Methodology Manual*. Institute for Security and Open Methodologies (ISECOM). https://www.isecom.org/OSSTMM.3.pdf

McGraw, G. (2006). *Software security: Building security in*. Addison-Wesley Professional.

MITRE Corporation. (2023). *MITRE ATT&CK® Enterprise Matrix (v13)*. https://attack.mitre.org/

OWASP Foundation. (2020). *Web Security Testing Guide (WSTG) - Version 4.2*. Open Web Application Security Project. https://owasp.org/www-project-web-security-testing-guide/

Penetration Testing Execution Standard Team. (2014). *The Penetration Testing Execution Standard (PTES)*. http://www.pentest-standard.org/

Scarfone, K., Souppaya, M., Cody, A., & Orebaugh, A. (2008). *Technical guide to information security testing and assessment* (NIST Special Publication 800-115). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-115

Schneier, B. (2000). *Secrets and lies: Digital security in a networked world*. John Wiley & Sons.

Ware, W. H. (1970). *Security controls for computer systems: Report of Defense Science Board Task Force on Computer Security* (Report R-609-1). RAND Corporation. https://www.rand.org/pubs/reports/R609-1.html



Boulevard Tecnológico de Progreso SN x 62, Progreso, Yucatán,
C.P. 97320 Tel. 969 934 3023 E-mail: dir01_dprogreso@tecnm.mx
www.progreso.tecnm.mx